

Architecture proposée

L'architecture recommandée sépare clairement le frontend (Angular) et le backend (Express). Le client Angular et le serveur Express sont deux projets distincts, chacun avec son propre `package.json` et son dossier `node_modules` ¹. Par exemple, on peut structurer la solution ainsi :

```
frontend/                                # application Angular 21 + Angular Material
├── src/
│   ├── app/
│   │   ├── core/                        # services globaux, interceptors, guards,
│   │   │   │   modèles partagés
│   │   │   ├── shared/                  # composants, pipes, directives réutilisables,
│   │   │   │   modules Angular Material
│   │   │   ├── features/                # modules fonctionnels (ex. auth, contrats,
│   │   │   │   etc.)
│   │   │   ├── app-routing.module.ts
│   │   │   └── app.module.ts           # module racine
│   └── assets/
├── angular.json
└── package.json

backend/                                  # serveur Express en TypeScript
├── src/
│   ├── config/                          # configuration (CORS, clés, base de données
│   │   │   lowdb, etc.)
│   ├── controllers/                     # gestionnaires des routes REST (req, res)
│   ├── routes/                          # définition des routes (authentification,
│   │   │   contrats...)
│   ├── middlewares/                     # middleware personnalisés (vérification JWT,
│   │   │   gestion erreurs...)
│   ├── services/                        # logique métier (authentification, gestion des
│   │   │   contrats...)
│   ├── models/                          # interaction avec la base lowdb (ex. modèles
│   │   │   User, Contract)
│   ├── utils/                           # fonctions utilitaires (hash de mot de passe,
│   │   │   générateur de token, etc.)
│   ├── app.ts                           # création et configuration de l'app Express
│   └── server.ts                         # lancement du serveur HTTP
├── db.json                              # fichier de données LowDB (stocke utilisateurs,
│   │   │   contrats...)
├── package.json
└── tsconfig.json
```

Dans cet agencement, Angular utilise son arborescence standard (`src/app` pour le code, avec des modules fonctionnels) ² ³. On adopte une **architecture par « feature modules »** : chaque

fonctionnalité (authentification, gestion des contrats...) est isolée dans son propre module Angular ³ . Le backend Express suit les bonnes pratiques : on sépare clairement la configuration (`config/`), les routes, les contrôleurs, les middleware et les services métier ⁴ ⁵ . Par exemple, les requêtes passent d'abord par le middleware CORS, puis par les middlewares d'authentification JWT et enfin par les contrôleurs métier, comme le résume BezKoder ⁵ .

Communication Angular ↔ Express (REST)

La communication se fait via HTTP REST. Côté Angular, on utilise le service `HttpClient` et on paramètre l'URL de base de l'API dans `environment.ts`. On implémente souvent un **HTTP Interceptor** : il intercepte chaque requête sortante pour y ajouter l'en-tête `Authorization: Bearer <access_token>`, et gère les erreurs 401 pour éventuellement déclencher un rafraîchissement de token ⁶ . Côté Express, on active CORS globalement (`app.use(cors())`) afin d'autoriser les requêtes cross-origin depuis l'UI Angular ⁷ . (Par défaut `app.use(cors())` gère automatiquement les requêtes *preflight* OPTIONS ⁷ .) On définit des routes REST claires, par exemple `/api/auth/signup`, `/api/auth/signin`, `/api/auth/refresh` pour l'authentification, et `/api/contracts` pour accéder aux contrats métier. Chaque requête au backend doit porter le JWT dans l'en-tête (sauf inscription/connexion). Angular peut aussi utiliser un proxy de développement (`proxy.conf.json`) pour simplifier les appels API pendant le dev.

Authentification JWT et gestion des tokens

On met en place un système d'authentification **maison** avec JWT (jetons d'accès et de rafraîchissement). Le backend Express gère l'inscription (`/auth/signup`), la connexion (`/auth/signin`), le rafraîchissement (`/auth/refresh`) et la déconnexion. Après connexion, le serveur renvoie un **access token** (de courte durée) et un **refresh token** (plus long).

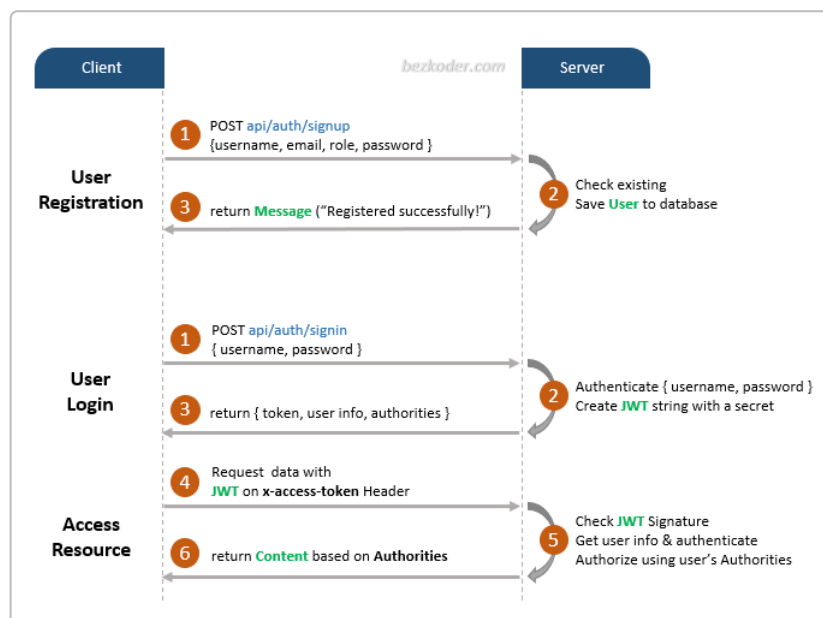


Figure : Workflow d'authentification JWT – inscription, connexion et requêtes protégées (le JWT est envoyé dans l'en-tête `Authorization` des requêtes). Une fois connecté, chaque utilisateur récupère via l'API ses propres données (contrats) filtrées par `userId` .

Les bonnes pratiques de sécurité dictent une durée de vie courte pour le token d'accès (quelques minutes) et plus longue pour le refresh token (quelques jours) ⁸ ⁹. Par exemple, on peut avoir un access token valable 5-15 minutes et un refresh token valable 7 jours ⁹. Pour le stockage côté client : **ne pas conserver les tokens en clair dans** `localStorage` (vulnérable aux attaques XSS ¹⁰ ¹¹). L'approche la plus sûre est de garder l'access token en mémoire (dans un service Angular) ¹², et de stocker le refresh token dans un cookie HTTP seulement (HttpOnly+Secure) ¹³ ¹⁴. Les flags `HttpOnly` et `Secure` rendent le cookie inaccessible en JavaScript et forcé en HTTPS, et `SameSite` protège partiellement contre le CSRF ¹³. Ainsi, en cas de XSS, l'attaquant ne voit pas les tokens, et les requêtes avec cookie `HttpOnly` sont automatiquement gérées par le navigateur.

Le rafraîchissement du token se fait côté client typiquement via un interceptor : si une requête reçoit une réponse 401, on appelle `/auth/refresh` pour obtenir un nouveau access token et on réessaie la requête. Si le rafraîchissement échoue (token expiré), on redirige vers la page de login. À la déconnexion, on peut invalider côté serveur le refresh token (par exemple le supprimer de la base).

Stockage des données avec LowDB

Le backend utilise **lowdb**, une base de données JSON légère pour Node.js, adaptée aux petits projets. LowDB est « léger » et « TypeScript-friendly » avec des écritures atomiques sûres ¹⁵. On initialise par exemple une structure par défaut (users, contrats) et on utilise un adaptateur JSON (ex. `JSONFile`) pointant sur `db.json` ¹⁵. Exemples de schéma de données dans lowdb :

```
{
  "users": [
    {"id":1, "username":"alice", "passwordHash":"...", "refreshToken":"..."},
    {"id":2, "username":"bob", "passwordHash":"...", "refreshToken":"..."}
  ],
  "contracts": [
    {"id":10, "userId":1, "title":"Contrat A", "content":"..."},
    {"id":11, "userId":2, "title":"Contrat B", "content":"..."}
  ]
}
```

On accède aux données avec l'API de lowdb : par exemple `const db = new Low(new JSONFile('db.json')); await db.read();` puis `db.data?.contracts.filter(c => c.userId===userId)`. Après modification des données (ajout d'un user, création d'un contrat, etc.), on appelle `await db.write()` pour persister. LowDB simplifie le stockage (pas de setup de serveur de BD) ¹⁶ ¹⁵. Bien sûr, on doit **hasher les mots de passe** (ex. `bcrypt`) avant de les stocker. Chaque contrat contient un champ `userId` pour s'assurer, lors des requêtes protégées, que seul l'utilisateur propriétaire y accède.

Bonnes pratiques supplémentaires

- **Organisation du code Angular** : utiliser la CLI pour générer des modules et composants, structurer avec des modules `core` (services globaux, interceptors) et `shared` (composants/pipes utilitaires) ³ ².
- **Sécurité des tokens** : ne jamais stocker les tokens sensibles dans du stockage web accessible (XSS), préférer les cookies `HttpOnly` et la mémoire vive pour les tokens ¹⁰ ¹³. Régénérer l'access token avant son expiration pour éviter les interruptions (stratégie de rafraîchissement).

- **Communication Angular/Express** : utiliser `HttpClient` (avec interceptor) sur Angular, activer CORS sur Express (`app.use(cors())`) ⁷ , et formater les routes REST de manière cohérente (`/api/auth` , `/api/contracts` , etc.).
- **LowDB en TypeScript** : profiter du typage en définissant des interfaces pour les données (ex. `User` , `Contract`) et en configurant `Low<YourSchema>` ¹⁵ . Versionner ou ignorer le fichier `db.json` selon le besoin (en dev on peut l'ignorer sous nodemon pour éviter les boucles infinies ¹⁷).

Ce découpage clair (dossiers frontend/backend) et ces pratiques garantissent un projet modulaire, maintenable et sécurisé, tout en répondant aux besoins fonctionnels (authentification JWT, accès aux contrats propres à chaque utilisateur, utilisation de lowdb pour le stockage JSON).

Sources : Nos recommandations s'appuient sur la documentation et des retours d'expérience (Angular style guide ³ , Express best practices ⁴ , articles sur JWT ¹⁴ ¹⁰ ⁸ ⁹ , documentation LowDB ¹⁵ ¹⁶ , etc.).

-
- ¹ node.js - Angular and Express - File structure - Stack Overflow
<https://stackoverflow.com/questions/49503580/angular-and-express-file-structure>
 - ² File structure • Angular
<https://angular.dev/reference/configs/file-structure>
 - ³ typescript - Angular project structure best practice - Stack Overflow
<https://stackoverflow.com/questions/52933476/angular-project-structure-best-practice>
 - ⁴ Best Practices for Structuring an Express.js Project - DEV Community
<https://dev.to/moibra/best-practices-for-structuring-an-expressjs-project-148i>
 - ⁵ JWT Authentication with Node.js Express, Angular 11 example - BezKoder
<https://www.bezkoder.com/node-js-angular-11-jwt-authentication/>
 - ⁶ ¹⁴ JWT in Angular: Automatic Token Refresh & Best Practices | by Niklas Wockenfuß | Medium
<https://medium.com/@n.wocke/jwt-in-angular-automatic-token-refresh-best-practices-b54827ea050c>
 - ⁷ Express cors middleware
<https://expressjs.com/en/resources/middleware/cors.html>
 - ⁸ JWT Security Best Practices: Checklist for APIs | Curity
<https://curity.io/resources/learn/jwt-best-practices/>
 - ⁹ The Struggle: Managing Access and Refresh Tokens in Web Apps | by Eric Abell | Medium
https://medium.com/@eric_abell/the-struggle-managing-access-and-refresh-tokens-in-web-apps-1bd70a3a6f01
 - ¹⁰ ¹² reactjs - Is it safe and convenient to store jwt token in Localstorage or cookies and to store the user's data in Context API - Stack Overflow
<https://stackoverflow.com/questions/71815202/is-it-safe-and-convenient-to-store-jwt-token-in-localstorage-or-cookies-and-to-s>
 - ¹¹ ¹³ JWT Storage in React: Local Storage vs Cookies Security Battle
<https://cybersierra.co/blog/react-jwt-storage-guide/>
 - ¹⁵ GitHub - typicode/lowdb: Simple and fast JSON database
<https://github.com/typicode/lowdb>
 - ¹⁶ ¹⁷ Building a CRUD app with Node, Express and LowDB (Beginner) | by Dennis Grosch | Medium
<https://headty.medium.com/building-a-crud-app-with-node-express-and-lowdb-beginner-cec2d5d1b65e>